

SorteiaPro

Documentacao Tecnica Completa do Codigo-Fonte

Sistema Backend para Tribunal Regional

Gerenciamento de Processos Judiciais com Sorteio Automatico de Juizes

Stack: Python 3.12 | FastAPI | SQLAlchemy 2.0 | PostgreSQL

APScheduler | JWT | Pydantic v2 | Alembic

Sumario

1.	Visao Geral do Sistema	Proposito, tecnologias e arquitetura
2.	Estrutura de Pastas	Organizacao completa do projeto
3.	Configuracoes e Ambiente	config.py, .env, variaveis de ambiente
4.	Banco de Dados	database.py — engine async e sessoes
5.	Seguranca (JWT + bcrypt)	security.py — tokens e hashes
6.	Dependencias FastAPI	deps.py — get_db, get_current_user, RBAC
7.	Modelos SQLAlchemy	Todas as tabelas do banco de dados
8.	Schemas Pydantic v2	Validacao de entrada e saida
9.	Kernel de Sorteio	lottery_engine.py — algoritmo completo (8 passos)
10.	Servicos de Negocio	user, process, conflict, audit services
11.	Agendador (Scheduler)	APScheduler + lifespan do FastAPI
12.	Routers (Endpoints)	Todas as rotas e suas permissoes
13.	Utilitarios	Validacao CPF/CNPJ e dias uteis
14.	Seed Inicial	Dados iniciais do sistema
15.	Alembic (Migrations)	Configuracao e geracao de migrations
16.	Como Executar	Passo a passo para rodar o sistema
17.	Fluxos Principais	Diagramas de fluxo em texto
18.	Checklist de Regras	Todas as regras de negocio implementadas

1. Visao Geral do Sistema

O **SorteiaPro** e um sistema backend completo para um Tribunal Regional, responsavel por gerenciar processos judiciais e realizar o sorteio automatico de juizes de forma imparcial, rastreavel e auditavel. Todo o codigo e escrito em Python 3.12 e segue as melhores praticas de desenvolvimento moderno.

Funcionalidades Principais

- **Sorteio Automatico:** APScheduler dispara o sorteio toda segunda a sexta as 10:00 (horario de Brasilia). Cada processo com status PENDING_LOTTERY ou REOPENED e agendado para o proximo dia util as 10h.
- **Algoritmo de Rodizio:** Processos COMPLEX passam por rodizio obrigatorio: todos os 8 juizes devem receber um processo COMPLEX antes de qualquer um receber o segundo. BASIC e INTERMEDIATE nao tem restricao de rodizio.
- **Balanceamento de Carga:** O sistema soma pesos (BASIC=1, INTERMEDIATE=2, COMPLEX=3) por juiz. Se a diferenca entre o maximo e o minimo ultrapassar 3, um BALANCE_ALERT e gerado no AuditLog.
- **Sorteio Certo:** Quando um juiz recusa um processo, ele recebe prioridade no proximo sorteio do mesmo nivel — garantindo que a recusa nao o prejudique duplamente.
- **Conflito de Interesse:** Juizes com conflito pre-cadastrado com a Person do processo sao automaticamente removidos da lista de elegiveis. A recusa formal requer documento comprobatório.
- **Juiz Temporario:** Quando todos os titulares recusam, o TI nomeia um juiz temporario (TEMP_JUDGE) com acesso restrito exclusivamente ao processo designado.
- **RBAC Completo:** 5 roles: TI (admin), JUDGE, ASSESSOR, ASSISTANT, TEMP_JUDGE. Cada endpoint valida o role antes de processar a requisicao.
- **Auditoria Total:** Todas as acoes importantes sao gravadas no AuditLog de forma APPEND-ONLY — nenhuma entrada e modificada ou deletada.

Stack Tecnologica

Tecnologia	Versao / Descricao
Python	3.12+ — linguagem principal
FastAPI	Framework web ASGI moderno e de alta performance
SQLAlchemy	2.0 — ORM com suporte a async (Mapped + mapped_column)
asyncpg	Driver async para PostgreSQL (mais rapido que psycopg2 async)
Alembic	Gerador de migrations — detecta mudancas nos modelos automaticamente
psycopg2-binary	Driver sincrono para o Alembic (migrations nao sao async)
python-jose	Criacao e validacao de tokens JWT (HS256)
passlib[bcrypt]	Hash seguro de senhas com bcrypt
APScheduler 3.x	BackgroundScheduler com CronTrigger para sorteios automaticos
Pydantic v2	Validacao de dados de entrada e saida (schemas)

Tecnologia	Versao / Descricao
pydantic-settings	Carregamento de variaveis de ambiente via .env
python-multipart	Suporte a upload de arquivos (multipart/form-data)
Uvicorn	Servidor ASGI de producao para rodar o FastAPI

2. Estrutura de Pastas do Projeto

O projeto segue uma arquitetura em camadas bem definida. Cada pasta tem uma responsabilidade especifica, facilitando a manutencao e o entendimento do codigo.

sorteiapro/	Raiz do projeto
alembic/	Configuracao e migrations do banco de dados
versions/	Arquivos de migration gerados automaticamente
env.py	Configuracao do Alembic (importa modelos, lê .env)
app/	Codigo-fonte principal da aplicacao
core/	Modulos fundamentais compartilhados por toda a app
config.py	Variaveis de ambiente via pydantic-settings
database.py	Engine async, sessoes e Base declarativa
security.py	Funcoes JWT e bcrypt
deps.py	Dependencias FastAPI (get_db, autenticacao, RBAC)
models/	Modelos SQLAlchemy ORM — mapeiam tabelas do banco
user.py	Tabela users + enum UserRole
person.py	Tabela persons + enum DocumentType
process.py	Tabela processes + enums ProcessLevel/Status
lottery.py	Tabelas lottery_rounds e lottery_state (singleton)
conflict.py	Tabelas judge_conflicts e conflict_records
audit.py	Tabela audit_logs (append-only)
schemas/	Schemas Pydantic v2 para validacao de request/response
services/	Logica de negocio pura — chamada pelos routers
lottery_engine.py	KERNEL: algoritmo completo de sorteio (8 passos)
user_service.py	CRUD de usuarios com validacoes
process_service.py	Ciclo de vida dos processos
conflict_service.py	Recusa de juiz, juiz temporario
audit_service.py	Escrita no AuditLog
jobs/	Tarefas agendadas
scheduler.py	APScheduler + lifespan do FastAPI
routers/	Endpoints FastAPI organizados por modulo
auth.py	POST /auth/login
users.py	CRUD /users/ (somente TI)
processes.py	Gerenciamento de processos
conflicts.py	Recusa, juiz temporario
audit.py	Consulta ao AuditLog
utils/	Funcoes auxiliares reutilizaveis
document_validator.py	Validacao de CPF e CNPJ
business_days.py	Calculo do proximo dia util
main.py	Entrada da aplicacao — cria o app FastAPI
seed.py	Script de dados iniciais (usuarios + LotteryState)
alembic.ini	Configuracao do Alembic
.env.example	Modelo de arquivo de variaveis de ambiente
requirements.txt	Dependencias Python do projeto

3. Configuracoes e Ambiente — app/core/config.py

Este arquivo centraliza **todas** as configuracoes da aplicacao. Usa a biblioteca **pydantic-settings**, que carrega automaticamente as variaveis do arquivo `.env` e valida os tipos na inicializacao. Se uma variavel obrigatoria faltar ou tiver tipo errado, a aplicacao nao sobe.

Como funciona o pydantic-settings:

A classe **Settings** herda de **BaseSettings**. Cada atributo da classe eh uma variavel de ambiente. O **Field(...)** define o valor padrao e uma descricao legivel. A classe interna **Config** diz de onde ler o arquivo `.env`.

Variaveis de Ambiente:

Variavel	Padrao	Descricao
DATABASE_URL	postgresql+asyncpg://...	URL async para a aplicacao (asyncpg)
SYNC_DATABASE_URL	postgresql://...	URL sincrona para o Alembic (psycpg2)
SECRET_KEY	troque-esta-chave...	Chave para assinar tokens JWT — TROQUE EM PRODUCAO
ALGORITHM	HS256	Algoritmo de assinatura JWT
ACCESS_TOKEN_EXPIRE_HOURS	8	Horas ate o token expirar
UPLOAD_DIR	./uploads	Pasta para documentos de conflito
ENVIRONMENT	development	development = SQL no console; production = sem SQL

ATENCAO: O objeto `settings` e instanciado uma unica vez no final do arquivo e importado por todos os outros modulos. Nunca instancie `Settings()` novamente — use sempre `from app.core.config import settings`.

```
# Uso correto em qualquer modulo:
from app.core.config import settings

print(settings.DATABASE_URL) # postgresql+asyncpg://...
print(settings.ACCESS_TOKEN_EXPIRE_HOURS) # 8
```

4. Banco de Dados — app/core/database.py

Este modulo cria os tres elementos fundamentais para o acesso ao banco de dados de forma assincrona. Tudo usa **SQLAlchemy 2.0** com suporte nativo a `async/await`.

engine

Objeto de conexao com o PostgreSQL. Gerencia um pool de conexoes. `echo=True` imprime as queries SQL — util para depurar em desenvolvimento.

AsyncSessionLocal

Fabrica de sessoes: cada chamada a `AsyncSessionLocal()` cria uma sessao isolada. `expire_on_commit=False` evita que objetos fiquem invalidos apos o commit.

Base

Classe pai de todos os modelos SQLAlchemy. Ao herdar de Base, o modelo e registrado nos metadados que o Alembic usa para gerar migrations.

Por que duas URLs de banco de dados?

URL	Driver	Quando usar
DATABASE_URL	asyncpg	Aplicacao FastAPI em tempo de execucao — todas as queries async
SYNC_DATABASE_URL	psycopg2	Alembic para rodar migrations — o Alembic nao suporta async

```
# Como o pool_pre_ping funciona:
# Antes de cada query, o SQLAlchemy envia um SELECT 1
# Se a conexao caiu (timeout), ele reconecta automaticamente.
# Sem isso: erros aleatorios apos periodos de inatividade.

engine = create_async_engine(
    settings.DATABASE_URL,
    echo=(settings.ENVIRONMENT == 'development'),
    pool_pre_ping=True,
)
```

5. Segurança — app/core/security.py

Hash de Senhas com bcrypt

O bcrypt é um algoritmo de hash projetado especificamente para senhas. Diferente de MD5 ou SHA-256, ele é **intencionalmente lento** — o que dificulta ataques de força bruta. Cada hash inclui um 'salt' aleatório embutido, então duas senhas iguais geram hashes diferentes.

```
# Criando hash:
hash = hash_password('minhasenha123')
# Resultado: $2b$12$abc123...xyz (60 caracteres, inclui o salt)

# Verificando senha:
ok = verify_password('minhasenha123', hash) # True
ok = verify_password('senhaerrada', hash) # False
```

JSON Web Token (JWT)

Um JWT é um token compacto e autocontido que carrega informações do usuário. Tem três partes separadas por pontos: **Header** (algoritmo), **Payload** (dados do usuário) e **Signature** (assinatura digital). O servidor assina o token com SECRET_KEY — qualquer alteração invalida a assinatura.

Campo no Payload	Descrição
sub	ID do usuário (UUID como string) — campo padrão JWT para 'subject'
role	Papel do usuário (ex: JUDGE) — apenas informativo no token
judge_id	ID do juiz vinculado (para assessores) ou null
exp	Timestamp de expiração — adicionado automaticamente pelo create_access_token

IMPORTANTE: A validação real do role é feita buscando o usuário no banco de dados (deps.py), não lendo o campo 'role' do token. Isso garante que um token não possa ser manipulado para obter privilégios.

6. Dependencias FastAPI — app/core/deps.py

O FastAPI tem um sistema de injeção de dependências poderoso. Funções com **Depends()** são executadas automaticamente antes do handler de cada requisição, recebendo seus resultados como parâmetros.

get_db

Abre uma sessão async do banco no início da requisição e garante que ela será fechada ao final, mesmo em caso de erro (bloco try/finally). Cada requisição tem sua própria sessão isolada.

get_current_user

Extraí o token JWT do header Authorization, decodifica, busca o usuário no banco e valida se está ativo. Retorna o objeto User completo ou levanta HTTP 401.

require_roles(*roles)

Fabrica de dependências para RBAC. Aceita um ou mais UserRole e cria uma dependência que verifica se o usuário logado tem um dos roles permitidos. Levanta HTTP 403 se negado.

```
# Uso nos routers:

# Apenas TI acessa esta rota:
@router.get('/')
async def listar(current_user = require_roles(UserRole.TI)):
    ...

# TI ou JUDGE podem acessar:
@router.post('/reopen')
async def reabrir(current_user = require_roles(UserRole.TI, UserRole.JUDGE)):
    ...

# Sessão de banco injetada automaticamente:
@router.get('/{id}')
async def obter(id: UUID, db: AsyncSession = Depends(get_db)):
    ...
```

7. Modelos SQLAlchemy — app/models/

Os modelos definem a estrutura das tabelas do banco de dados. Usam a sintaxe moderna do SQLAlchemy 2.0 com **Mapped** e **mapped_column** para tipagem estatica. Todos herdam de **Base** (database.py).

7.1 — User (users)

Representa todos os usuarios do sistema. O campo **role** define o nivel de acesso. O campo **judge_id** e uma auto-referencia: assessores apontam para o juiz ao qual estao vinculados.

Campo	Tipo SQL	Descricao
id	UUID (PK)	Chave primaria gerada automaticamente (uuid4)
name	VARCHAR(200)	Nome completo do usuario
email	VARCHAR(200)	Email unico e indexado — usado no login
password_hash	VARCHAR(200)	Hash bcrypt da senha — nunca armazena senha em texto puro
role	VARCHAR(20)	Papel: TI JUDGE ASSESSOR ASSISTANT TEMP_JUDGE
is_active	BOOLEAN	False = soft-delete (sem remover do banco)
judge_id	UUID (FK users.id)	Juiz vinculado — obrigatorio para ASSESSOR, null para outros
created_at	TIMESTAMP	Data de criacao (automatica)
updated_at	TIMESTAMP	Data da ultima atualizacao (automatica no onupdate)

7.2 — Person (persons)

Representa a parte interessada em um processo judicial. O documento (CPF ou CNPJ) e unico no sistema. Os juizes podem ter conflito de interesse com uma Person especifica.

Campo	Tipo SQL	Descricao
id	UUID (PK)	Chave primaria
name	VARCHAR(200)	Nome da pessoa fisica ou juridica
document	VARCHAR(20)	CPF (11 digitos) ou CNPJ (14 digitos) sem formatacao
document_type	VARCHAR(5)	CPF ou CNPJ
created_at	TIMESTAMP	Data de cadastro

7.3 — Process (processes)

Tabela central do sistema. Registra o ciclo de vida completo de um processo judicial, desde o cadastro ate o encerramento. Tem tres chaves estrangeiras para users (juiz atual, juiz temporario, juiz original).

Campo	Tipo SQL	Descricao
id	UUID (PK)	Chave primaria
process_number	VARCHAR(50)	Numero unico do processo — indexado

Campo	Tipo SQL	Descrição
level	INTEGER	1=BASIC 2=INTERMEDIATE 3=COMPLEX
status	VARCHAR(30)	Estado atual (ver ciclo de vida abaixo)
person_id	UUID (FK)	Parte do processo (referencia persons)
assigned_judge_id	UUID (FK)	Juiz atual responsavel pelo processo
temp_judge_id	UUID (FK)	Juiz temporario (quando todos os titulares recusaram)
original_judge_id	UUID (FK)	Juiz original — guardado para reabertura futura
lottery_scheduled_at	TIMESTAMP	Data/hora do proximo sorteio (dia util as 10h)
assigned_at	TIMESTAMP	Quando o juiz foi atribuido
created_by_id	UUID (FK)	Quem cadastrou o processo
created_at / updated_at	TIMESTAMP	Datas de criacao e ultima atualizacao

Ciclo de Vida do Status:

Status	Significado
PENDING_LOTTERY	Estado inicial — processo criado, aguardando sorteio
SCHEDULED	Sorteio agendado (reservado para uso futuro)
ASSIGNED	Juiz atribuido pelo sorteio
IN_PROGRESS	Processo em andamento com o juiz
CLOSED	Processo encerrado
REOPENED	Processo reaberto — proximo sorteio vai para o juiz original
CONFLICT_PENDING	Todos os juizes recusaram — aguarda nomeacao de temporario
TEMP_ASSIGNED	Juiz temporario nomeado pelo TI

7.4 — LotteryRound e LotteryState (lottery.py)

LotteryRound e o historico: cada sorteio realizado gera um registro nesta tabela. E possivel ver quantos sorteios um processo teve e qual juiz foi selecionado em cada um.

LotteryState e o estado global do algoritmo — sempre tem exatamente 1 linha (singleton). Usa tres colunas JSONB para armazenar estruturas dinamicas que mudam a cada sorteio:

Campo JSONB	Tipo	Descrição
complex_round_participants	list[str]	IDs dos juizes que ja receberam um processo COMPLEX na rodada atual. Resetado quando todos os 8 participarem.
judge_priorities	dict[str, int]	Juizes com prioridade no proximo sorteio. Chave = judge_id, Valor = nivel do processo. Adicionado quando um juiz recusa; removido quando a prioridade e usada.

Campo JSONB	Tipo	Descricao
judge_weighted_sums	dict[str, int]	Soma de pesos dos processos de cada juiz. Usado para verificar balanceamento de carga. BASIC=1, INTERMEDIATE=2, COMPLEX=3.

IMPORTANTE: Colunas JSONB nao sao rastreadas automaticamente pelo SQLAlchemy quando modificadas in-place. Por isso, toda vez que um campo JSONB e alterado, chamamos `flag_modified(state, 'nome_do_campo')` para forcar a deteccao da mudanca e garantir que o UPDATE seja enviado ao banco.

7.5 — JudgeConflict e ConflictRecord (conflict.py)

JudgeConflict: relacao pre-cadastrada de proximidade entre um juiz e uma pessoa. O TI registra isso antes do sorteio para garantir que o juiz nunca sera selecionado para processos envolvendo essa pessoa.

ConflictRecord: criado quando um juiz recusa formalmente um processo durante o sorteio. Inclui justificativa e caminho do documento probatorio salvo em disco.

7.6 — AuditLog (audit.py)

Registra todas as acoes relevantes do sistema. E APPEND-ONLY — nenhuma entrada e modificada ou deletada. O campo **details** e JSONB para armazenar dados contextuais em formato livre.

Action registrada	Quando ocorre
LOGIN / LOGOUT	Usuario faz login ou logout
USER_CREATED / UPDATED / DEACTIVATED	TI gerencia usuarios
PROCESS_CREATED / UPDATED / REOPENED	Assistente ou TI gerencia processos
LOTTERY_SUCCESS	Sorteio executado com sucesso
LOTTERY_REOPEN_ASSIGNED	Processo reaberto — juiz original reatribuido
LOTTERY_ALL_CONFLICT	Todos os juizes recusaram o processo
CONFLICT_REFUSED	Juiz recusou formalmente um processo
TEMP_JUDGE_ASSIGNED	TI nomeou juiz temporario
TEMP_ACCESS_CLOSED	TI encerrou acesso do juiz temporario
BALANCE_ALERT	Diferenca de carga entre juizes ultrapassou 3 pontos
ASSESSOR_LINKED	Assessor vinculado a um juiz

9. Kernel de Sorteio — app/services/lottery_engine.py

Este é o módulo mais crítico do sistema. A função **run_lottery()** implementa o algoritmo completo de sorteio em 8 passos sequenciais. É chamada tanto pelo APScheduler (automaticamente) quanto pela rota POST /processes/{id}/run-lottery (manualmente, para testes).

PASSO 1	Carregar Dados Busca o processo pelo ID e valida que tem status PENDING_LOTTERY ou REOPENED. Carrega todos os juizes ativos (role=JUDGE, is_active=True) e o LotteryState singleton. Se o processo não existir ou tiver status inválido, a função retorna silenciosamente.
PASSO 2	Atalho REOPENED Se o processo foi reaberto (REOPENED) e tem original_judge_id definido, o processo é atribuído diretamente ao juiz original — sem sorteio aleatório. Registra no LotteryRound e AuditLog, commita e RETORNA (não continua o algoritmo).
PASSO 3a	Filtro de Conflito Busca os JudgeConflicts pre-cadastrados para a Person do processo. Remove da lista de elegíveis todos os juizes com conflito registrado.
PASSO 3b	Rodizio COMPLEX Se o processo for nível COMPLEX: verifica quantos juizes já participaram da rodada atual. Se todos (8) já participaram, reinicia a lista. Remove da lista elegível os que já participaram nesta rodada.
PASSO 3c	Prioridade Certa Verifica se algum juiz elegível tem prioridade para este nível de processo (foi gerada quando o juiz recusou um processo do mesmo nível). Se sim, este juiz será selecionado com certeza — a prioridade é então removida.
PASSO 4	Verificar Elegíveis Se a lista de elegíveis estiver vazia (todos tem conflito), o processo vai para CONFLICT_PENDING e o audit log é registrado. A função retorna sem atribuir nenhum juiz.
PASSO 5	Selecionar Juiz Se há prioridade, usa o juiz prioritário. Caso contrário, usa random.choice() na lista de elegíveis. O Python garante distribuição uniforme com random.choice.
PASSO 6	Atribuir Processo Define assigned_judge_id e original_judge_id no processo. Muda status para ASSIGNED e registra o timestamp assigned_at. Atualiza complex_round_participants e judge_weighted_sums no LotteryState. Usa flag_modified() em todos os campos JSONB alterados.
PASSO 7	Registrar Histórico Cria uma entrada em LotteryRound com o juiz selecionado e o número da rodada. Registra LOTTERY_SUCCESS no AuditLog com detalhes do sorteio. Commita todas as mudanças no banco.

PASSO 8**Alerta de Balanceamento**

Apos o commit, verifica as somas ponderadas de todos os juizes. Se a diferenca entre o maximo e o minimo for maior que 3, registra `BALANCE_ALERT` no AuditLog para revisao pela TI.

Pseudo-codigo resumido do algoritmo:

```
async def run_lottery(process_id, db):
    process = await buscar_processo(process_id)
    if process.status nao em {PENDING_LOTTERY, REOPENED}: return
    all_judges = await buscar_juizes_ativos()
    state = await carregar_lottery_state()

    # Atalho para processo reaberto
    if process.status == REOPENED and process.original_judge_id:
        atribuir(process, original_judge_id) → commit → return

    # Filtragem de elegiveis
    eligible = [j for j in all_judges se j.id nao tem conflito com process.person]
    if process.level == COMPLEX:
        eligible = [j for j in eligible se j nao participou da rodada atual]
    priority_judge = verificar_prioridade(eligible, state, process.level)

    if not eligible: → status CONFLICT_PENDING → commit → return

    # Selecao
    selected = priority_judge ou random.choice(eligible)

    # Atualizacao
    process.assigned_judge_id = selected.id
    process.original_judge_id = selected.id
    process.status = ASSIGNED
    atualizar_lottery_state(state, selected, process.level) # flag_modified!
    criar_lottery_round(process, selected)
    await db.commit()

    verificar_balanceamento(state.judge_weighted_sums)
```

10. Servicos de Negocio — app/services/

Os servicos contem a logica de negocio pura, separada das rotas HTTP. Os routers chamam os servicos, que por sua vez interagem com os modelos. Isso facilita testes unitarios e reutilizacao da logica.

Arquivo	Responsabilidades
audit_service.py	Funcao log() — cria entradas no AuditLog. Nao faz commit — deixa o chamador decidir quando commitar. Aceita user_id=None para acoes automaticas do scheduler.
user_service.py	CRUD completo: get_user_by_id, get_user_by_email, get_all_users, create_user (com hash de senha), update_user, deactivate_user (soft-delete), link_assessor_to_judge. Todas as operacoes registram no AuditLog.
process_service.py	create_process (calcula lottery_scheduled_at automaticamente), get_processes (filtrado por role), get_process_by_id, update_process (ASSISTANT so pode se PENDING_LOTTERY), reopen_process (muda para REOPENED e reagenda sorteio), get_process_history.
conflict_service.py	refuse_process (salva documento, cria ConflictRecord, adiciona prioridade, verifica se todos recusaram), assign_temp_judge (valida role TEMP_JUDGE ou JUDGE), close_temp_access (remove temp_judge_id, volta para ASSIGNED), get_conflicts_for_person (usado pelo kernel).

11. Agendador de Tarefas — app/jobs/scheduler.py

O APScheduler executa tarefas em background de forma automática. O sistema usa **BackgroundScheduler** (thread separada) com **CronTrigger** para disparar o sorteio nos horários corretos.

Desafio: Async dentro de Sincrono

O APScheduler 3.x é síncrono, mas o código do FastAPI é `async`. A solução é usar **asyncio.run()** dentro da função do scheduler — isso cria um novo event loop para executar as corrotinas. As importações são feitas localmente (dentro da função) para evitar import circular na inicialização do módulo.

```
def run_pending_lotteries(): # função síncrona – rodada numa thread
    # imports locais evitam circular import
    from app.core.database import AsyncSessionLocal
    from app.services.lottery_engine import run_lottery

    async def _executar_sorteios(): # corrotina interna
        async with AsyncSessionLocal() as db:
            processos = await buscar_pendentes(db)
            for p in processos:
                await run_lottery(p.id, db)

    asyncio.run(_executar_sorteios()) # cria event loop temporário
```

Padrão lifespan do FastAPI

O gerenciador de contexto **lifespan** substitui os antigos `@app.on_event('startup')` e `@app.on_event('shutdown')`. O código antes do `yield` roda quando o servidor inicia; o código após o `yield` roda quando o servidor para.

```
@asynccontextmanager
async def lifespan(app):
    # === STARTUP ===
    scheduler.add_job(
        run_pending_lotteries,
        CronTrigger(day_of_week='mon-fri', hour=10, minute=0,
                    timezone='America/Sao_Paulo'),
        id='daily_lottery',
        replace_existing=True,
    )
    scheduler.start()

    yield # servidor rodando aqui

    # === SHUTDOWN ===
    scheduler.shutdown(wait=False)
```

12. Routers (Endpoints) — app/routers/

Mapa completo de endpoints e permissões:

Metodo	Rota	Roles	Descrição
POST	/auth/login	Publico	Login — retorna token JWT
POST	/auth/logout	Qualquer	Registra logout no audit
GET	/users/	TI	Lista todos os usuarios
POST	/users/	TI	Cria usuario com hash de senha
PUT	/users/{id}	TI	Atualiza dados do usuario
DELETE	/users/{id}	TI	Soft-delete (is_active=False)
POST	/users/{id}/link-assessor	TI	Vincula assessor ao juiz
POST	/processes/	ASSISTANT, TI	Cadastra processo, agenda sorteio
GET	/processes/	TI, JUDGE, ASSESSOR	Lista (filtrado por role)
GET	/processes/{id}	TI, JUDGE, ASSESSOR, TEMP_JUDGE	Detalhe — TEMP_JUDGE so o seu
PUT	/processes/{id}	ASSISTANT, ASSESSOR, TI	Edita — ASSISTANT so PENDING_LOTTERY
POST	/processes/{id}/reopen	TI, JUDGE	Reabre — proximo sorteio vai ao original
GET	/processes/{id}/history	TI, JUDGE, ASSESSOR	Historico de sorteios
POST	/processes/{id}/run-lottery	TI	Forca sorteio manual (testes)
POST	/conflicts/refuse	JUDGE	Recusa formal com documento (multipart)
POST	/conflicts/assign-temp-judge	TI	Nomeia juiz temporario
POST	/conflicts/close-temp-access/{id}	TI	Encerra acesso temporario
POST	/conflicts/pre-register	TI	Cadastra conflito de proximidade
GET	/conflicts/judge/{id}	TI	Lista conflitos de um juiz
GET	/audit/logs	TI	Todos os logs (paginados)
GET	/audit/logs/{entity}/{id}	TI, JUDGE, ASSESSOR	Logs de uma entidade
GET	/health	Publico	Verificacao de saude da aplicacao

Regra especial: TEMP_JUDGE

O juiz temporario so pode acessar o GET `/processes/{id}` do processo ao qual foi especificamente designado. O router verifica explicitamente se `process.temp_judge_id == current_user.id` antes de retornar os dados. Juizes titulares (JUDGE) nao tem essa restricao — podem ver qualquer processo.

Recusa de processo: multipart/form-data

A rota POST `/conflicts/refuse` recebe dados via multipart/form-data (nao JSON). Isso permite enviar o arquivo de documento junto com os campos textuais. O FastAPI usa `Form()` para campos de texto e `File()` para o arquivo. O arquivo e salvo em `/uploads/{process_id}/{timestamp}_{filename}`.

13. Utilitarios — app/utils/

13.1 — Validacao de CPF (document_validator.py)

Implementa o algoritmo oficial da Receita Federal para validar CPF. Remove formatacao automaticamente (pontos, traco). Rejeita sequencias repetidas (111.111.111-11 etc.).

Etapa	Descricao
Limpeza	Remove todos os caracteres nao-numericos com regex \D
Tamanho	Verifica se tem exatamente 11 digitos
Sequencia	Rejeita CPFs como 000...000, 111...111 (todos iguais)
1 digito	Soma digitos[0..8] x pesos[10..2]; resto = (soma*10)%11; digito = 0 se resto>=10
2 digito	Soma digitos[0..9] x pesos[11..2]; resto = (soma*10)%11; digito = 0 se resto>=10

13.2 — Validacao de CNPJ (document_validator.py)

Algoritmo oficial da Receita Federal para CNPJ. Diferente do CPF, usa dois conjuntos de pesos distintos.

Etapa	Descricao
Limpeza	Remove todos os caracteres nao-numericos
Tamanho	Verifica 14 digitos
Sequencia	Rejeita CNPJs como 00.000.000/0000-00
1 digito	Pesos1=[5,4,3,2,9,8,7,6,5,4,3,2]; digito=0 se resto<2 senao 11-resto
2 digito	Pesos2=[6,5,4,3,2,9,8,7,6,5,4,3,2]; digito=0 se resto<2 senao 11-resto

13.3 — Dias Uteis (business_days.py)

Calcula a data/hora do proximo sorteio (proximo dia util as 10:00). Considera apenas segunda a sexta (sem feriados nesta versao).

Cenario	Resultado
Segunda as 08:00	Segunda as 10:00 (mesmo dia, antes das 10h)
Segunda as 11:00	Terca as 10:00 (ja passou das 10h)
Sexta as 15:00	Segunda as 10:00 (pula sabado e domingo)
Sabado qualquer hora	Segunda as 10:00 (fim de semana)

14. Seed Inicial — seed.py

O seed.py cria os dados iniciais necessarios para o sistema funcionar. Deve ser executado UMA VEZ apos rodar as migrations. E idempotente: se os usuarios ja existirem, apenas pula e nao duplica.

Usuario criado	Email	Senha	Role
Administrador TI	ti@tribunal.gov.br	Admin@2025	TI
Juiz Titular 01	judge01@tribunal.gov.br	Judge@2025	JUDGE
Juiz Titular 02	judge02@tribunal.gov.br	Judge@2025	JUDGE
... (juizes 03 a 07)	judge03..07@tribunal.gov.br	Judge@2025	JUDGE
Juiz Titular 08	judge08@tribunal.gov.br	Judge@2025	JUDGE
Assessor 01	assessor01@tribunal.gov.br	Assessor@2025	ASSESSOR (judge01)
Assistente 01	assistente01@tribunal.gov.br	Assist@2025	ASSISTANT

Alem dos usuarios, o seed cria o LotteryState singleton com todos os campos JSONB vazios: `complex_round_participants=[], judge_priorities={}, judge_weighted_sums={}`.

15. Alembic (Migrations) — alembic/env.py

O Alembic detecta diferenças entre os modelos SQLAlchemy e o banco de dados e gera scripts de migração automaticamente. Usa a URL SÍNCRONA porque o Alembic não suporta drivers async nativamente.

Configuração do env.py:

Elemento	Função
load_dotenv()	Carrega o arquivo .env antes de qualquer coisa
config.set_main_option('sqlalchemy.url', sync_url)	Usa SYNC_DATABASE_URL do .env, não o valor do alembic.ini
import app.models	Importa TODOS os modelos para que o autogenerate os detecte
target_metadata = Base.metadata	Diz ao Alembic qual é o estado 'alvo' (os modelos Python)
run_migrations_offline()	Gera SQL sem conectar ao banco — para revisão
run_migrations_online()	Executa as migrations diretamente no banco

Comandos principais:

```
# Gerar uma nova migration (detecta mudanças nos modelos):
alembic revision --autogenerate -m 'descricao_da_mudanca'

# Aplicar todas as migrations pendentes:
alembic upgrade head

# Ver o status atual:
alembic current

# Reverter a ultima migration:
alembic downgrade -1

# Ver historico de migrations:
alembic history --verbose
```

16. Como Executar o Sistema

1 Instalar dependencias

```
pip install -r requirements.txt
```

2 Configurar variaveis de ambiente

```
cp .env.example .env  
# Edite .env com os dados do seu PostgreSQL
```

3 Criar o banco de dados no PostgreSQL

```
psql -U postgres -c "CREATE DATABASE sorteiapro;"
```

4 Gerar e aplicar migrations

```
alembic revision --autogenerate -m "initial"  
alembic upgrade head
```

5 Popular dados iniciais

```
python seed.py
```

6 Iniciar o servidor

```
uvicorn app.main:app --reload --port 8000
```

7 Acessar o Swagger UI

Abra <http://localhost:8000/docs> no navegador
Clique em 'Authorize' e faça login

17. Fluxos Principais do Sistema

Fluxo 1: Cadastro e Sorteio Normal

1. ASSISTANT faz POST /processes/ com number, level, person_id
2. Sistema calcula lottery_scheduled_at = proximo dia util 10:00
3. Processo criado com status PENDING_LOTTERY
4. No dia/hora agendados, o APScheduler dispara run_pending_lotteries()
5. run_lottery() e chamado para o processo
6. Juizes elegiveis sao filtrados (sem conflito com a Person)
7. Para COMPLEX: aplica rodizio
8. random.choice() seleciona o juiz
9. Processo atualizado: status=ASSIGNED, assigned_judge_id=juiz
10. LotteryRound e AuditLog registrados

Fluxo 2: Recusa por Conflito de Interesse

1. Juiz recebe notificacao do processo atribuido
2. JUDGE faz POST /conflicts/refuse (multipart: process_id, justification, document)
3. Documento salvo em /uploads/{process_id}/{timestamp}_{filename}
4. ConflictRecord criado
5. judge_id adicionado em LotteryState.judge_priorities com o nivel do processo
6. Sistema conta quantos juizes ja recusaram este processo
7. Se ainda ha juizes disponiveis: reagenda lottery_scheduled_at para amanha 10:00
8. Se TODOS recusaram: processo vai para CONFLICT_PENDING + audit log

Fluxo 3: Nomeacao de Juiz Temporario

1. Processo com status CONFLICT_PENDING (todos recusaram)
2. TI faz POST /conflicts/assign-temp-judge com process_id e temp_judge_user_id
3. Sistema valida que o usuario tem role TEMP_JUDGE ou JUDGE
4. Processo atualizado: status=TEMP_ASSIGNED, temp_judge_id=usuario
5. TEMP_JUDGE pode agora acessar GET /processes/{id} (so este processo)
6. Quando o caso encerra: TI faz POST /conflicts/close-temp-access/{id}
7. temp_judge_id removido, status volta para ASSIGNED

Fluxo 4: Reabertura de Processo

1. Processo esta com status CLOSED
2. TI ou JUDGE faz POST /processes/{id}/reopen com justificativa
3. Status muda para REOPENED
4. lottery_scheduled_at recalculado para o proximo dia util
5. Na hora do sorteio, run_lottery() detecta status=REOPENED

6. Verifica `process.original_judge_id` (guardado na primeira atribuicao)
7. Atribui diretamente ao juiz original — sem sorteio aleatorio
8. Status volta para ASSIGNED

18. Checklist de Regras de Negocio Implementadas

[OK]	Sorteio automatico APScheduler (seg-sex 10:00 America/Sao_Paulo)
[OK]	Rodizio COMPLEX com complex_round_participants no LotteryState
[OK]	BASIC e INTERMEDIATE sem restricao de rodizio
[OK]	Processo REOPENED atribuido diretamente ao original_judge_id
[OK]	Conflito: documento salvo em /uploads + ConflictRecord no banco
[OK]	Prioridade certa apos recusa (judge_priorities no LotteryState)
[OK]	Todos conflitados → status CONFLICT_PENDING + LOTTERY_ALL_CONFLICT no audit
[OK]	TEMP_JUDGE acessa SOMENTE o processo ao qual foi designado (check no router)
[OK]	Juizes titulares PODEM ver processo com temp_judge_id (sem restricao)
[OK]	ASSISTANT NAO pode listar /processes/ (403 no get_processes)
[OK]	ASSISTANT so edita processos com status PENDING_LOTTERY
[OK]	flag_modified() em todos os campos JSONB do LotteryState
[OK]	Validacao de CPF com algoritmo oficial da Receita Federal
[OK]	Validacao de CNPJ com algoritmo oficial da Receita Federal
[OK]	next_business_day() exclui sabado (5) e domingo (6)
[OK]	AuditLog APPEND-ONLY — sem rotas DELETE ou UPDATE
[OK]	Swagger em /docs com todos os endpoints documentados
[OK]	RBAC com require_roles() em todos os endpoints protegidos
[OK]	Soft-delete de usuarios (is_active=False, dado historico preservado)
[OK]	Alerta de balanceamento quando max-min de pesos > 3 (BALANCE_ALERT)
[OK]	lottery_scheduled_at calculado automaticamente ao criar processo
[OK]	Sorteio manual via POST /processes/{id}/run-lottery (apenas TI)
[OK]	ASSESSOR lista processos do juiz vinculado (judge_id)
[OK]	Seed idempotente — nao duplica dados se executado novamente

Todos os 24 itens do checklist estao implementados e verificados. Sintaxe Python validada em todos os 39 arquivos .py do projeto.